

 SAKARYA UNİVERSİTESİ	T.C. SAKARYA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ UYGULAMALI MÜHENDİSLİK DENEYİMİ EĞİTİMİ (UMDE)	
---	---	--

UMDE ADAY MÜHENDİS FAALİYET ARA RAPORU (F-PG-1)

Aday Mühendisin Adı ve Soyadı	ATAKAN ÇALIŞKAN
UMDE İşletmesinin Adı	TEI-TUSAŞ MOTOR SANAYİİ A.Ş.
Çalıştığı Bölüm ve Alan	Yazılım Müdürlüğü – Gömülü Yazılım Geliştirme
Teslim Edilen Ara Rapor No	4
Faaliyet Zaman Aralığı	09 / 03 /2026 - 12 / 03 /2026

UMDE Aday Mühendis; faaliyet ara raporunu danışman öğretim üyesi/görevlisine her hafta teslim etmek zorundadır. Akademik danışman bu rapora göre değerlendirme yapacaktır.

FAALİYET ARA RAPORU
<u>GENEL C DİLİ ÇALIŞMALARI</u> Bu hafta gömülü yazılım geliştirme sürecinde ihtiyaç duyulan temel C programlama yapıları üzerine yoğunlaşmıştır. Yapılan çalışmalarda öncelikle değişken tanımı ve scope kavramı ele alınmış; local, global ve static değişken yapılarının bellek davranışı ve erişim sınırları değerlendirilmiştir. Özellikle local değişkenlerin fonksiyon bloğu ile sınırlı olduğu, global değişkenlerin program genelinde erişilebilirlik sağladığı ve static tanımlamaların bellek ömrünü uzattığı konuları incelenmiştir. Bunun yanında fonksiyon yapısı, parametre geçiş mantığı, geri dönüş değeri kullanımı ve kodun modüler hale getirilmesi açısından fonksiyonel parçalama yaklaşımı üzerinde durulmuştur. Temel veri tipleri tarafında int, float ve char gibi standart türlerin yanı sıra stdint.h kütüphanesi ile gelen uint8_t, uint16_t, uint32_t ve int8_t gibi sabit uzunluklu veri tipleri değerlendirilmiştir. Bu yapıların özellikle gömülü sistemlerde register genişlikleri ve veri taşınabilirliği açısından önemli olduğu görülmüştür. Kontrol yapıları kapsamında for ve while döngüleri, switch-case ve break yapısı ve karar mekanizmaları tekrar edilmiştir. Ayrıca overflow davranışının, özellikle sınırlı bit genişliğine sahip veri tiplerinde sayısal taşma oluşturduğu ve bunun donanım seviyesinde sonuçları etkileyebileceği gözlemlenmiştir. Pointer konusu kapsamında, pointer'ın doğrudan verinin kendisini değil bellek adresini tuttuğu, * ve & operatörleri ile adres ve içerik erişiminin sağlandığı çalışılmıştır. Pointer-array ilişkisi incelenerek dizi elemanlarına pointer aritmetiği üzerinden erişim mantığı değerlendirilmiştir. Son olarak stack ve heap bellek alanları arasındaki fark, const anahtar sözcüğünün veri güvenliği

açısından rolü, **volatile** kullanımının derleyici optimizasyonunu sınırlayarak donanım tarafından değişebilen verilerin doğru okunmasını sağlaması ve bit düzeyinde **set/clear/toggle/check** işlemlerinin register kontrolündeki önemi üzerinde durulmuştur. Bu tekrar çalışmaları, kart üzerindeki çevre birimlerinin register seviyesinde kontrol edilmesi açısından doğrudan altyapı sağlamıştır.

Kod Geliştirme Aşamasına Geçiş

Bu aşamada, önceki haftalarda teorik olarak incelenen I2C haberleşme yapısı ve TMP117 sıcaklık sensörüne ait register organizasyonu, çalışan bir gömülü yazılım uygulamasına dönüştürülmüştür. Bu hafta geliştirilen kod yapısında, öncelikle sistemin temel çalışma mantığını doğrulamak amacıyla üretici tarafından sağlanan hazır kütüphaneler ve Driverlib fonksiyonları kullanılmıştır. Bu yaklaşım sayesinde ilk aşamada donanımın doğru şekilde çalıştırılması, sensörle haberleşmenin kurulması ve sıcaklık verisinin başarıyla okunması hedeflenmiştir.

Bununla birlikte mevcut yazılım yapısı, proje ilerledikçe daha modüler ve yeniden kullanılabilir bir mimariye dönüştürülecektir. İlerleyen haftalarda kodun **Driver – BSP – HAL** katmanlarına ayrılması planlanmaktadır. Bu yapı sayesinde donanıma özgü register işlemleri ile uygulama seviyesindeki fonksiyonlar birbirinden ayrılacak, kodun okunabilirliği ve bakım yapılabilirliği artacak, ayrıca aynı yazılım altyapısının farklı çevre birimleri veya farklı donanım platformlarında tekrar kullanılabilmesi mümkün hale gelecektir. Özellikle sürdürülebilirlik, test edilebilirlik ve geliştirilebilirlik açısından bu katmanlı mimari büyük önem taşımaktadır.

I2C ile Sıcaklık Ölçümü İçin Yazılımda İzlenen Genel Akış

TMP117 sıcaklık sensöründen veri okunabilmesi için yazılım tarafında belirli bir işlem sırasının takip edilmesi gerekmektedir. Bu akış, **HLR** benzeri üst seviye fonksiyonel mantıkla aşağıdaki şekilde tanımlanmıştır:

1. **Sistem başlatma işlemlerinin yapılması**
Mikrodenetleyicinin temel saat, sistem ve GPIO altyapısı aktif hale getirilir.
2. **I2C çevre birimi saatinin açılması**
Kullanılacak I2C modülüne ait peripheral clock enable edilerek çevre birimi erişilebilir hale getirilir.
3. **SDA ve SCL pinlerinin I2C fonksiyonuna atanması**
İlgili GPIO pinleri alternatif fonksiyon moduna alınarak I2C veri ve saat hattı olarak yapılandırılır.

4. **Pin elektriksel özelliklerinin ayarlanması**
I2C hattına uygun olacak şekilde pull-up ve asenkron giriş yapısı seçilir.
5. **I2C modülünün disable/reset durumunda yapılandırılması**
Yapılandırma güvenli şekilde yapılabilsin diye modül önce disable edilir.
6. **Master mod, haberleşme hızı ve adresleme yapısının ayarlanması**
I2C modülü 100 kHz hızda, 7-bit adresleme ve 8-bit veri genişliği ile çalışacak şekilde başlatılır.
7. **FIFO yapısının etkinleştirilmesi**
Veri alma-gönderme sırasında tamponlama yapısının kullanılması sağlanır.
8. **I2C modülünün aktif hale getirilmesi**
Konfigürasyon tamamlandıktan sonra modül enable edilerek çalışmaya hazır duruma getirilir.
9. **Hedef sensör adresinin belirlenmesi**
TMP117 sensörünün I2C adresi seçilir.
10. **Veri alma modunun seçilmesi ve okunacak byte sayısının tanımlanması**
Sensörden kaç byte veri alınacağı belirlenir.
11. **START condition üretilmesi ve veri alımının başlatılması**
I2C haberleşmesi fiziksel olarak başlatılır.
12. **Gelen verinin kontrol edilmesi ve okunması**
FIFO doluluk durumu kontrol edilerek üst ve alt byte okunur.
13. **STOP condition üretilmesi**
Haberleşme çevrimi düzgün şekilde sonlandırılır.
14. **Ham verinin sıcaklık değerine dönüştürülmesi**
Sensörden gelen iki byte birleştirilir ve fiziksel sıcaklık değeri hesaplanır.
15. **Güvenilirlik amaçlı kontrol mekanizmalarının uygulanması**
Sonsuz bekleme durumlarını önlemek için timeout yapısı ve yazılımın sürekli çalıştığını göstermek amacıyla loop sayacı kullanılır.

Include ve Global Değişkenler Kısmı

Kodun başlangıç bölümünde kullanılan **device.h** ve **driverlib.h** başlık dosyaları, mikrodenetleyicinin sistem başlatma işlemleri ile GPIO ve I2C gibi çevre birimlerinin kontrolüne yönelik hazır fonksiyonları projeye dahil etmektedir. Bu yapı sayesinde ilk doğrulama aşamasında doğrudan register adresleri ile çalışmak yerine, daha okunabilir ve hızlı geliştirilebilir bir yazılım yaklaşımı izlenmiştir.

Global değişkenler bölümünde yer alan hamVeri değişkeni sensörden okunan iki byte'lık sıcaklık bilgisinin birleştirilmiş halini saklamak için, **anlikSicaklik** değişkeni bu ham veriden elde edilen fiziksel sıcaklık değerini tutmak için, **loopSayaci** değişkeni ise yazılımın ana döngü içerisinde sürekli çalıştığını kontrol etmek amacıyla kullanılmıştır. Bu değişkenler aynı zamanda debug sürecinde izleme ve doğrulama kolaylığı sağlamaktadır.

Main Fonksiyonu

<u>Kod Satırı</u>	<u>Ne İşe Yarar?</u>	<u>Etkilediği Register / Alt Birim</u>
<code>uint16_t ustByte = 0;</code>	Sensörden okunacak üst byte verisini geçici olarak tutar.	RAM üzerinde yerel değişken
<code>uint16_t altByte = 0;</code>	Sensörden okunacak alt byte verisini geçici olarak tutar.	RAM üzerinde yerel değişken
<code>Device_init();</code>	Mikrodenetleyicinin temel sistem başlatma işlemlerini gerçekleştirir.	Sistem clock ve temel cihaz altyapısı
<code>Device_initGPIO();</code>	GPIO altyapısını başlatır ve pin yapılandırmasına hazır hale getirir.	GPIO alt sistemi
<code>SysCtl_enablePeripheral(SYSCTL_PERIPH_CLK_I2CB);</code>	I2C-B çevre biriminin saatini açar. Clock açılmadan modüle erişilemez.	System Control / Peripheral Clock Gating
<code>GPIO_setPinConfig(GPIO_40_I2CB_SDA);</code>	40 numaralı pini I2C-B SDA hattına yönlendirir.	GPIO mux registerları
<code>GPIO_setPadConfig(40, GPIO_PIN_TYPE_PULLUP);</code>	SDA hattı için pull-up yapısını ayarlar.	GPIO pad config registerları
<code>GPIO_setQualificationMode(40, GPIO_QUAL_ASYNC);</code>	SDA pini için asenkron örnekleme seçer.	GPIO qualification registerları
<code>GPIO_setPinConfig(GPIO_41_I2CB_SCL);</code>	41 numaralı pini I2C-B SCL hattına yönlendirir.	GPIO mux registerları
<code>GPIO_setPadConfig(41, GPIO_PIN_TYPE_PULLUP);</code>	SCL hattı için pull-up yapısını ayarlar.	GPIO pad config registerları
<code>GPIO_setQualificationMode(41, GPIO_QUAL_ASYNC);</code>	SCL pini için asenkron örnekleme seçer.	GPIO qualification registerları
<code>I2C_disableModule(I2CB_BASE);</code>	I2C modülünü yapılandırma öncesi disable duruma alır.	I2CMDR
<code>I2C_initMaster(I2CB_BASE, DEVICE_SYSCLOCK_FREQ, 100000, I2C_DUTYCYCLE_50);</code>	I2C modülünü master modda 100 kHz hat frekansı ile başlatır.	I2CPSC, I2CCLKL, I2CCLKH, I2CMDR
<code>I2C_setBitCount(I2CB_BASE, I2C_BITCOUNT_8);</code>	Veri genişliğini 8 bit olarak ayarlar.	I2CMDR içindeki bit count alanı

I2C_setAddressMode(I2CB_BASE, I2C_ADDR_MODE_7BITS);	7 bit adresleme modunu seçer.	I2C adresleme kontrol alanı
I2C_enableFIFO(I2CB_BASE);	FIFO yapısını etkinleştirir.	I2CFFTX, I2CFFRX
I2C_enableModule(I2CB_BASE);	I2C modülünü aktif hale getirir.	I2CMDR

While Döngüsü

<u>Kod Satırı</u>	<u>Ne İşe Yarar?</u>	<u>Etkilediği Register / Alt Birim</u>
I2C_setSlaveAddress(I2CB_BASE, 0x48);	Haberleşilecek sensörün adresini belirler.	I2CSAR
I2C_setConfig(I2CB_BASE, I2C_MASTER_RECEIVE_MODE);	Modülü master receive moduna geçirir.	I2CMDR
I2C_setDataCount(I2CB_BASE, 2);	Sensörden 2 byte veri okunacağını belirtir.	I2CCNT
I2C_sendStartCondition(I2CB_BASE);	I2C haberleşmesini başlatmak için START condition üretir.	I2CMDR.STT
while(I2C_getRxFIFOStatus(I2CB_BASE) < 2)	RX FIFO içinde 2 byte veri oluşana kadar bekler.	I2CFFRX durum bilgisi
ustByte = I2C_getData(I2CB_BASE);	Gelen ilk byte'ı okur.	I2CDRR
altByte = I2C_getData(I2CB_BASE);	Gelen ikinci byte'ı okur.	I2CDRR
I2C_sendStopCondition(I2CB_BASE);	Haberleşmeyi sonlandırmak için STOP condition üretir.	I2CMDR.STP
hamVeri = (ustByte << 8) altByte	Üst Byte'ı 8 birim sola kaydırarak alt byte'ı ekler.	Üst ve alt byte'ı birleştirerek 16 bit ham veriyi oluşturur.
anlikSicaklik = (float)((int16_t)hamVeri) * 0.0078125;	Ham veriyi signed 16 bit olarak yorumlayıp sıcaklık değerine dönüştürür.	RAM üzerinde yazılım işlemi
loopSayaci++;	Döngünün çalıştığını göstermek için sayaç artırılır.	RAM üzerinde yazılım işlemi
DEVICE_DELAY_US(100000);	Okumalar arasına yaklaşık 100 ms gecikme ekler.	Sistem delay altyapısı

Sonuç olarak bu hafta, TMP117 sıcaklık sensörü ile I2C haberleşmesinin kurulmasına yönelik temel gömülü yazılım çalışmaları tamamlanmış; sistem başlatma, pin konfigürasyonu, modül ayarları ve veri okuma akışı uygulama seviyesinde başarıyla gerçekleştirilmiştir. Elde edilen bu yapı, hem sensör verisinin güvenilir biçimde okunmasını sağlamış hem de ilerleyen haftalarda gerçekleştirilecek katmanlı sürücü mimarisi, hata yönetimi ve yazılım iyileştirme çalışmalarına temel oluşturmuştur.

 SAKARYA ÜNİVERSİTESİ	T.C. SAKARYA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ UYGULAMALI MÜHENDİSLİK DENEYİMİ EĞİTİMİ (UMDE)	
--	---	--

Aday Mühendisin İmzası	UMDE Sorumlusu İmzası	Danışman Öğretim Üyesi/Görevlisi İmzası